# Distributed Job Scheduler

Intern Assignment — Project Submission Report

**Candidate Name:** Sai Shivaram LN

**Registration Number:** RA231100304023

**Email:** ssrshivaram06@gmail.com

**Contact:** 7010003985

**Submission Date:** July 5, 2026

**GitHub Repository:** github.com/Ssr446/distributed-job-scheduler

**Live Deployment:** Dashboard — distributed-job-scheduler-dashboard.vercel.app (Vercel) ·

API — codity-backend.onrender.com (Render)

*Note: an uptime monitor pings the service periodically to reduce free-tier idle spin-down; an occasional cold start (~30-60s) is still possible.*

*Stack: Node.js / Express / Prisma / PostgreSQL / React (Vite) / Socket.IO*

# Table of Contents

Table of Contents....2
List of Figures....3
1. Executive Summary....4
2. System Architecture....5
2.1 Request flow for the core execution cycle....5
2.2 Authentication boundary....6
3. Database Design....7
3.1 Key design choices....7
3.2 Performance considerations....8
4. Backend Engineering....9
4.1 REST API....9
4.2 Authentication & security....9
4.3 Atomicity and idempotency....9
5. Reliability & Concurrency....10
5.1 Job lifecycle....10
5.2 Retry strategy....10
5.3 Dead Letter Queue....10
5.4 Concurrency safety....10
5.5 Testing evidence for this section....10
6. Frontend & UX....11
7. API Design....15
8. Testing....16
9. Deliverables & Design Decisions....17
9.1 Deliverables checklist....17
9.2 Selected design decisions....17
10. Bonus Features....18
11. Setup Instructions & Deployment....19
12. Known Limitations & Future Work....20
13. Conclusion....21

# List of Figures


| # | Figure | Section |
|---|---|---|
| 1 | System architecture and core execution-cycle sequence diagram. | Section 2 |
| 2 | Entity-relationship diagram of the full 16-model schema, grouped by domain: identity/org (blue), queue/retry (green), job core (amber), execution/reliability (purple), worker (teal), and security (red). | Section 3 |
| 3 | Dashboard — Overview | Section 6 |
| 4 | Job Explorer — filtered list with the execution log panel open | Section 6 |
| 5 | Queue detail — pause/resume control | Section 6 |
| 6 | Dead Letter Queue — entry detail with failure summary | Section 6 |
| 7 | Workers page — active worker with live heartbeat | Section 6 |
| 8 | Metrics page — throughput and status distribution | Section 6 |
| 9 | Terminal output of npm run test -w packages/api, all suites passing. | Section 8 |
| 10 | Live deployed dashboard, logged in, showing real data | Section 11 |

# 1. Executive Summary

This document describes the design, implementation, and current state of a distributed job scheduling platform built to the assignment specification: a system supporting authentication and multi-project job queues, configurable retry and concurrency policies, immediate/delayed/scheduled/recurring/batch job creation, atomic job claiming across concurrent workers, a full job lifecycle with retries and a Dead Letter Queue, and a live monitoring dashboard.

The submission prioritizes correctness of the core execution path — atomic claiming, idempotent state transitions, retry backoff, and DLQ escalation — over breadth of surface features, in line with the assignment's stated evaluation priorities. Where a feature is partially complete or a known limitation exists, it is documented explicitly in Section 12 rather than omitted, since an accurate account of engineering tradeoffs is itself part of what is being evaluated.

Section 2 onward mirrors the assignment's evaluation rubric, reproduced below, so each criterion can be located quickly.

| **Evaluation Criteria** | **Max Marks** | **Where addressed** |
|---|---|---|
| System Architecture | 20 | Section 2 |
| Database Design | 20 | Section 3 |
| Backend Engineering | 20 | Section 4 |
| Reliability & Concurrency | 15 | Section 5 |
| Frontend & UX | 10 | Section 6 |
| API Design | 5 | Section 7 |
| Documentation | 5 | Section 9 / Deliverables |
| Testing | 5 | Section 8 |

# 2. System Architecture

The system is a three-package monorepo: an API server (Express + Prisma), a standalone worker process, and a React dashboard. The API and worker communicate exclusively over HTTP — the worker is not a library imported into the API process, it is an independent client authenticated with its own service-account API key, distinct from the cookie-based JWT authentication used by the dashboard. This separation means the worker can, in principle, be scaled horizontally as an independent deployable unit without any change to the API.

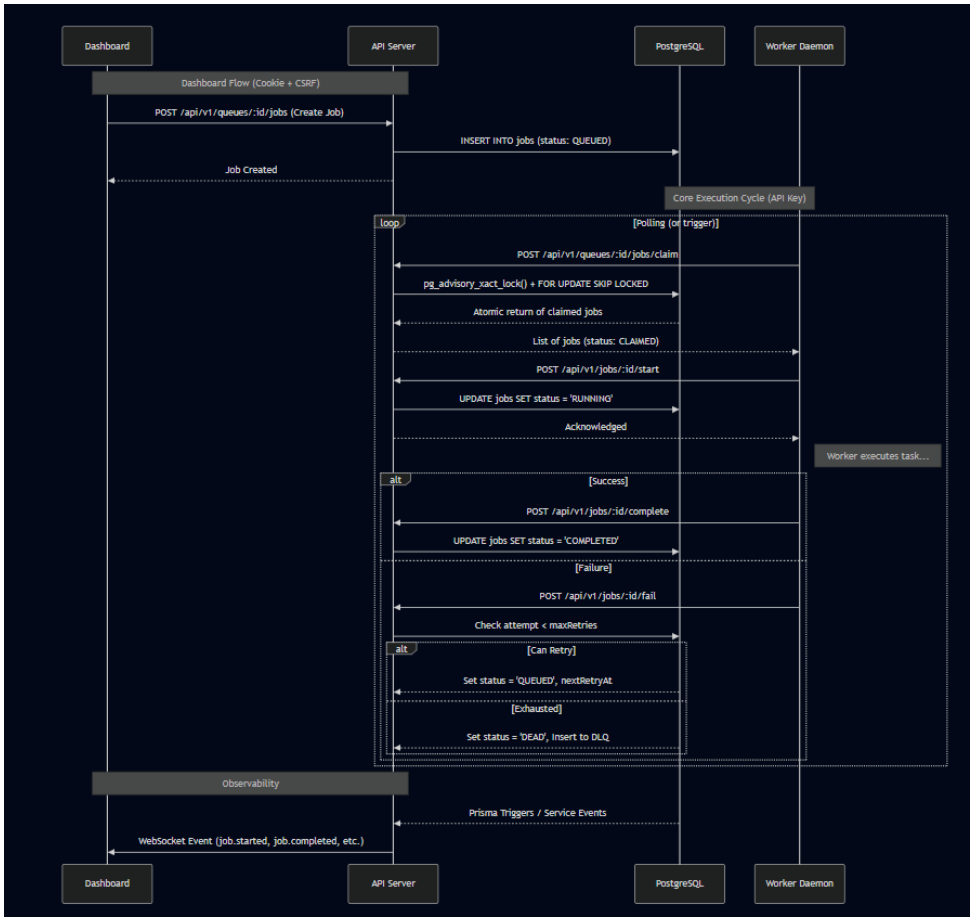



**Figure 1.** *System architecture and core execution-cycle sequence diagram.*

## 2.1 Request flow for the core execution cycle

- Worker polls POST /queues/:id/jobs/claim, authenticated via a dedicated service-account API key (SHA-256 hashed, constant-time compared) — never the dashboard's user JWT.
- Claiming is atomic: a PostgreSQL advisory lock scoped to the queue, combined with SELECT ... FOR UPDATE SKIP LOCKED, guarantees no two workers can claim the same job even under concurrent polling.
- Worker calls POST /jobs/:id/start to transition CLAIMED → RUNNING, executes a pluggable handler keyed by job type, then reports outcome via POST /jobs/:id/complete or /fail.
- Both endpoints are idempotent: a conditional update keyed on (id, expected status, claimedById) means a duplicate report from a retried HTTP call is a safe no-op rather than a double-write.
- On exhausted retries, the job is moved to DEAD and a Dead Letter Queue entry is created; dependent jobs waiting on it are cascaded to CANCELLED automatically.

- State changes are broadcast over a project-scoped WebSocket room so the dashboard reflects job status in real time without polling.

## 2.2 Authentication boundary

Two independent authentication schemes are deliberately kept separate rather than sharing one JWT-based path:

- Dashboard users authenticate via httpOnly, Secure cookies (JWT access + refresh, with rotation-on-use and DB-backed revocation by JTI), protected against CSRF via Origin/Referer validation on cookie-authenticated mutating requests.
- Workers authenticate via a separate API-key mechanism, routed so that worker-facing endpoints never pass through the cookie/CSRF middleware chain at all — the two traffic classes are structurally isolated, not just conditionally branched within shared middleware.

This separation was a deliberate design correction made during development after identifying that a shared authentication path created an unnecessary and hard-to-reason-about coupling between very differently-trusted callers. See Section 9 (Design Decisions) for the reasoning in full.

# 3. Database Design

The schema is implemented in PostgreSQL via Prisma and covers 16 models: User, Organization, OrgMembership, Project, Queue, RetryPolicy, Job, JobExecution, JobLog, JobDependency, ScheduledJob, Worker, WorkerHeartbeat, DeadLetterQueue, ApiKey, and RevokedToken (the latter two were added during the security-hardening pass covered in Section 9).

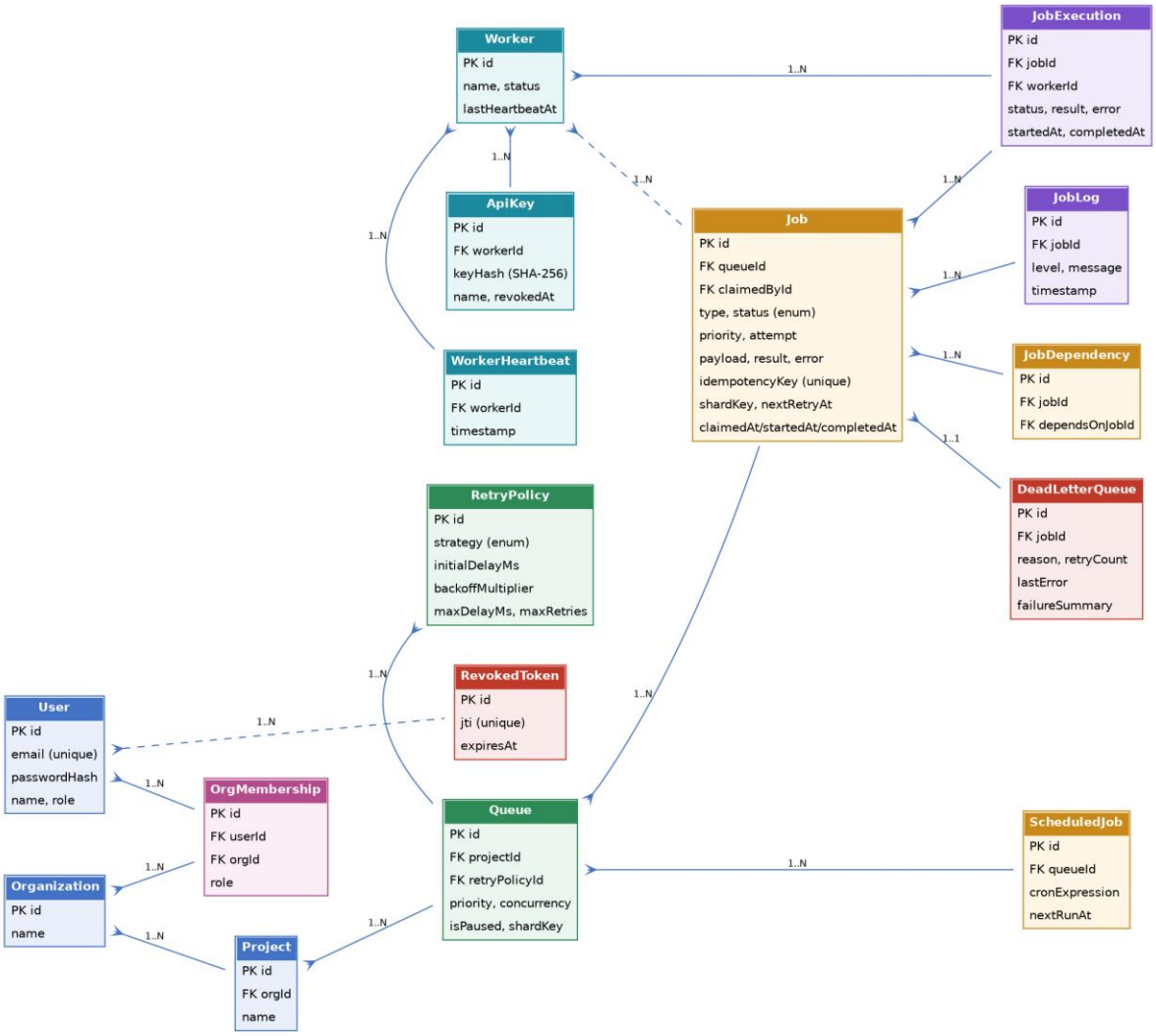



**Figure 2.** *Entity-relationship diagram of the full 16-model schema, grouped by domain: identity/org (blue), queue/retry (green), job core (amber), execution/reliability (purple), worker (teal), and security (red).*

## 3.1 Key design choices

- Primary keys are UUIDs throughout, avoiding sequential-ID enumeration and simplifying merges across the org/project/queue hierarchy.
- Foreign keys use explicit cascade rules matched to the ownership hierarchy (Organization → Project → Queue → Job → JobExecution/JobLog), so deleting an org cleanly removes everything beneath it without orphaned rows.

- The schema is normalized to 3NF; retry configuration lives once on RetryPolicy and is referenced by Queue rather than duplicated per job.
- Indexes are placed on the columns the claim query and metrics queries actually filter/sort on — (queueId, status), (workerId, timestamp) on heartbeats and logs — rather than indexing everything by default.
- Job carries both a status enum spanning the full lifecycle (QUEUED, SCHEDULED, WAITING, CLAIMED, RUNNING, COMPLETED, FAILED, DEAD, CANCELLED) and a nextRetryAt timestamp, so the claim query can filter retry-eligible jobs directly in SQL rather than in application code.
- An idempotencyKey unique constraint on Job allows safe request retries at job-creation time, independent of the execution-time idempotency handled by the claim/start/complete/fail state machine.
- ApiKey and RevokedToken were added post-launch to support service-account worker authentication and immediate JWT revocation respectively — both additive migrations, no changes to existing tables' shape.

## 3.2 Performance considerations

The claim query is a single atomic UPDATE ... WHERE id IN (SELECT ... FOR UPDATE SKIP LOCKED) statement rather than a read-then-write pair, avoiding a round trip and closing the race window entirely. Metrics endpoints use targeted raw SQL (parameterized via Prisma.sql, not string concatenation) for aggregate queries such as per-minute throughput, rather than pulling full row sets into application memory.

# 4. Backend Engineering

## 4.1 REST API

- Modular structure: each domain (auth, orgs, projects, queues, jobs, workers, DLQ, metrics) is a self-contained validator / service / controller / routes module.
- Input validation via Zod schemas on every mutating endpoint.
- Structured error handling via a centralized AppError type and global error-handling middleware, returning consistent JSON error shapes.
- Pagination and filtering on list endpoints (page/limit, status/type filters).
- Pino-based structured logging, including a per-request correlation ID surfaced in both logs and the response headers.

## 4.2 Authentication & security

| Feature | Status | Notes |
|---|---|---|
| httpOnly, Secure cookies for dashboard JWTs | Implemented | SameSite policy adapts to deployment topology — see Section 9.2 |
| Separate API-key auth for workers (SHA-256 + timing-safe compare) | Implemented | Worker traffic structurally isolated from cookie/CSRF middleware |
| CSRF protection (Origin/Referer validation) | Implemented | Scoped to cookie-authenticated requests only; bearer/API-key traffic exempt by design, not by header-sniffing |
| Token revocation (DB-backed, by JTI) + refresh rotation-on-use | Implemented | |
| Password policy (upper/lower/digit/special, min 8) | Implemented | |
| Rate limiting, with a dedicated higher-throughput limiter for worker polling | Implemented | Applied after authentication, not on unauthenticated header shape |
| Role-based access control (global + per-organization) | Implemented | requireRole / requireOrgRole middleware |
| Job-level ownership checks on retry/cancel | Implemented | Prevents a user acting on jobs outside their organization |

## 4.3 Atomicity and idempotency

Job claiming, starting, completing, and failing are each implemented as conditional updates guarded by both expected current state and caller identity (claimedById), so that concurrent or duplicated calls are safe no-ops rather than corrupting state. This was verified directly, not just asserted: an automated test issues concurrent/duplicate complete and fail calls against the same job and asserts exactly one JobExecution row, one JobLog entry, and one event emission result.

# 5. Reliability & Concurrency

## 5.1 Job lifecycle

QUEUED → (SCHEDULED / WAITING) → CLAIMED → RUNNING → COMPLETED, with a FAILED/DEAD branch governed by the queue's configured RetryPolicy.

## 5.2 Retry strategy

- Fixed, linear, and exponential backoff strategies, each configurable per queue via RetryPolicy (initial delay, multiplier, max delay, max retries).
- Backoff delay is computed from the pre-increment attempt count, so the first failure receives the first configured delay tier rather than skipping ahead.
- Up to 10% random jitter is added to every computed delay to avoid synchronized retry storms across many jobs failing at once.

## 5.3 Dead Letter Queue

- On exhausting max retries, the job transitions to DEAD and a DeadLetterQueue row is created recording the failure reason, retry count, and last error.
- Jobs waiting on a now-dead dependency are automatically cascaded to CANCELLED rather than waiting indefinitely.
- DLQ entries can be inspected and requeued from the dashboard, with a pattern-matched failure-category summary (timeout / network / validation / unknown) generated per entry.

## 5.4 Concurrency safety

- Atomic claiming via PostgreSQL advisory lock + FOR UPDATE SKIP LOCKED, verified under the worker's concurrent polling loop.
- Each worker enforces a configurable concurrency limit on simultaneously in-flight jobs.
- Graceful shutdown: both the API server and the worker catch SIGTERM/SIGINT; the worker specifically waits for in-flight jobs to finish (bounded by a timeout) before exiting, rather than dropping them mid-execution.

## 5.5 Testing evidence for this section

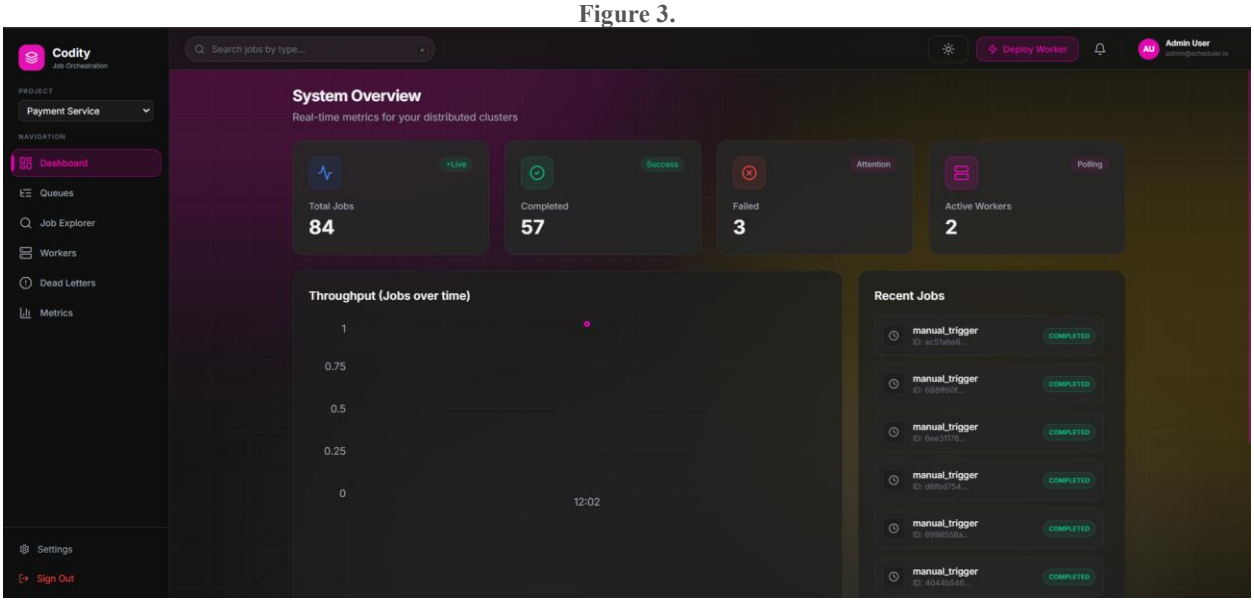
Automated tests exercise the full claim → start → fail → retry → fail → exhaust → DEAD path end to end against a real database, asserting exact row counts on JobExecution and DeadLetterQueue rather than only checking final status — see Section 8.

# 6. Frontend & UX

The dashboard is a React (Vite) single-page application covering queue configuration, job exploration, worker monitoring, dead-letter management, and system metrics, with real-time updates delivered over an authenticated, project-scoped WebSocket connection for status-change notifications.

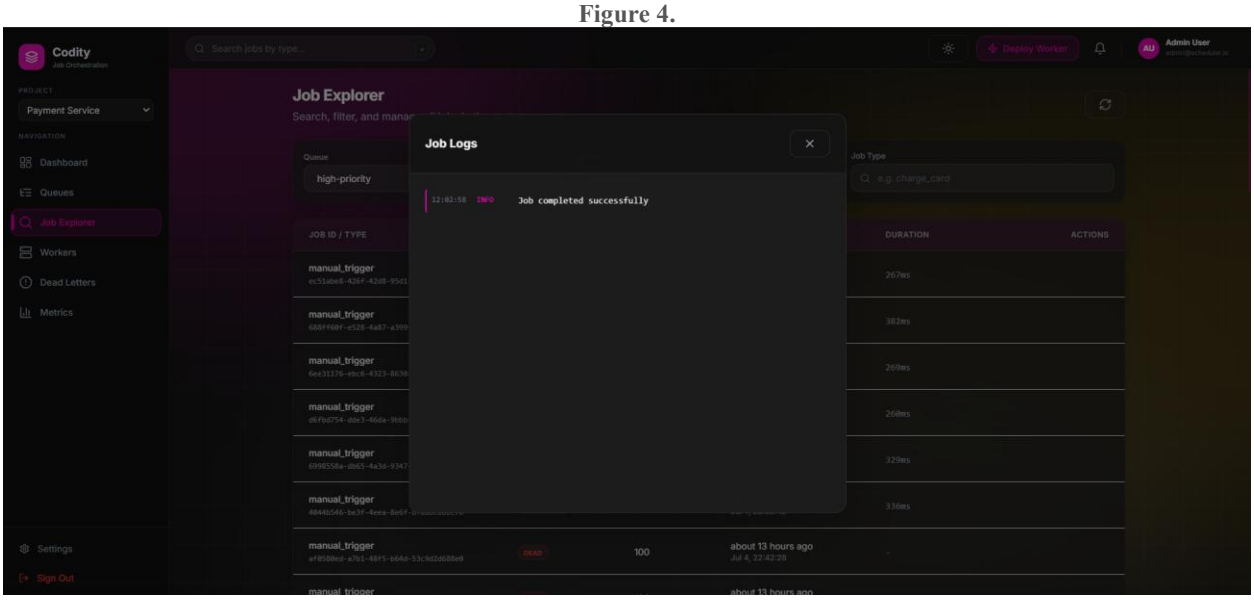
| Feature | Status | Notes |
|---|---|---|
| Multi-project switching | Implemented | Sidebar project selector; auto-selects a default project on first load |
| Dashboard overview (queue health, throughput) | Implemented | |
| Queue management (create, configure, pause/resume) | Implemented | |
| Job Explorer (filter by queue/status/type, per-job execution log viewer) | Implemented | See note below |
| Worker monitoring (status, heartbeat, active job count) | Implemented | |
| Dead Letter Queue view (inspect, requeue, failure summary) | Implemented | |
| Metrics page (throughput, latency, status distribution) | Implemented | |
| Real-time toast notifications via WebSocket on job state change | Implemented | |
| Job Explorer list auto-refresh on incoming events | Not yet implemented | The list currently updates via a manual refresh control rather than reacting to the same WebSocket events that drive toast notifications elsewhere. A cosmetic UX gap, not a functional one — see Section 12. |
| Mobile-responsive layout | Implemented, lightly audited | Sidebar collapses, tables scroll horizontally, touch targets adjusted; not exhaustively tested across devices |

**Figure 3.**



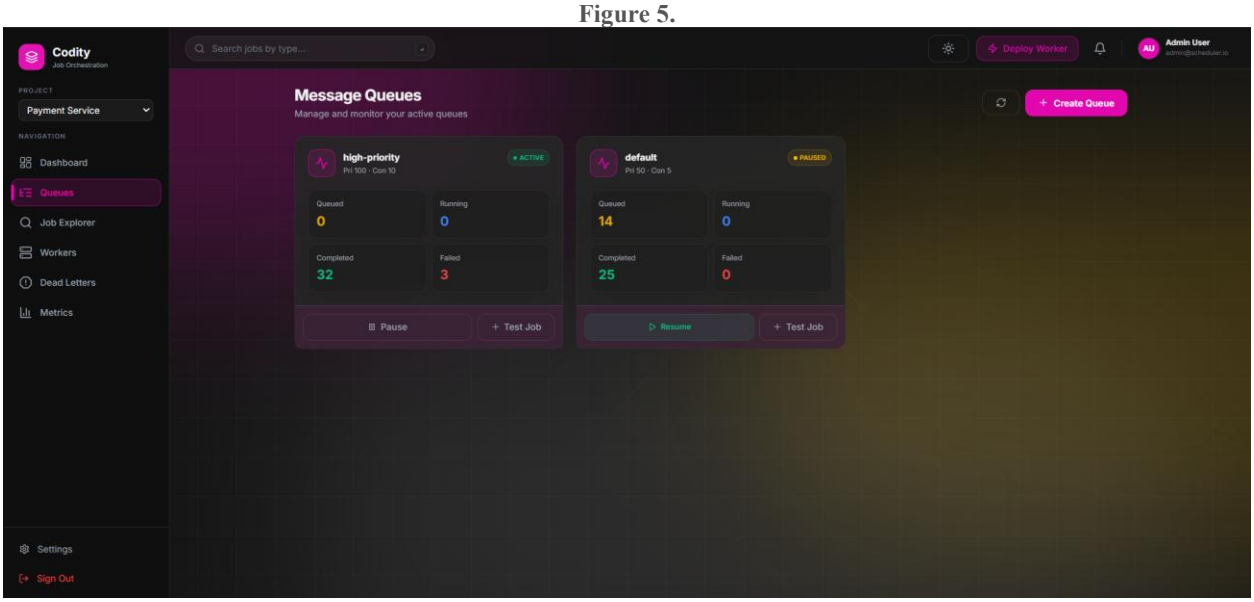

*Dashboard — Overview*

**Figure 4.**



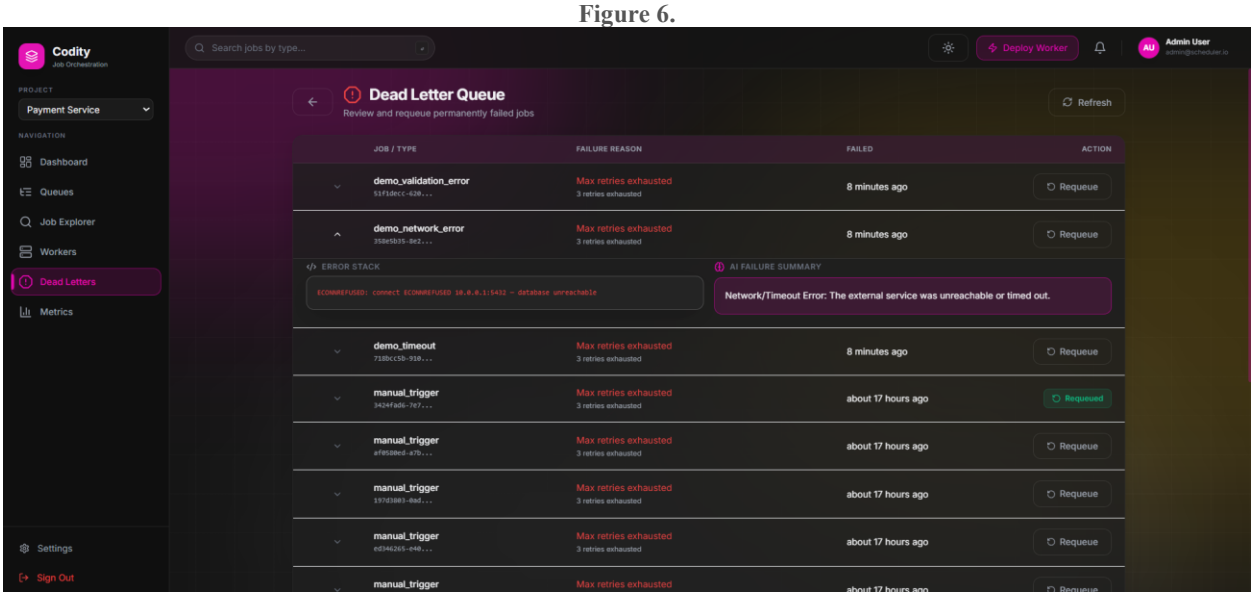

*Job Explorer — filtered list with the execution log panel open*

**Figure 5.**



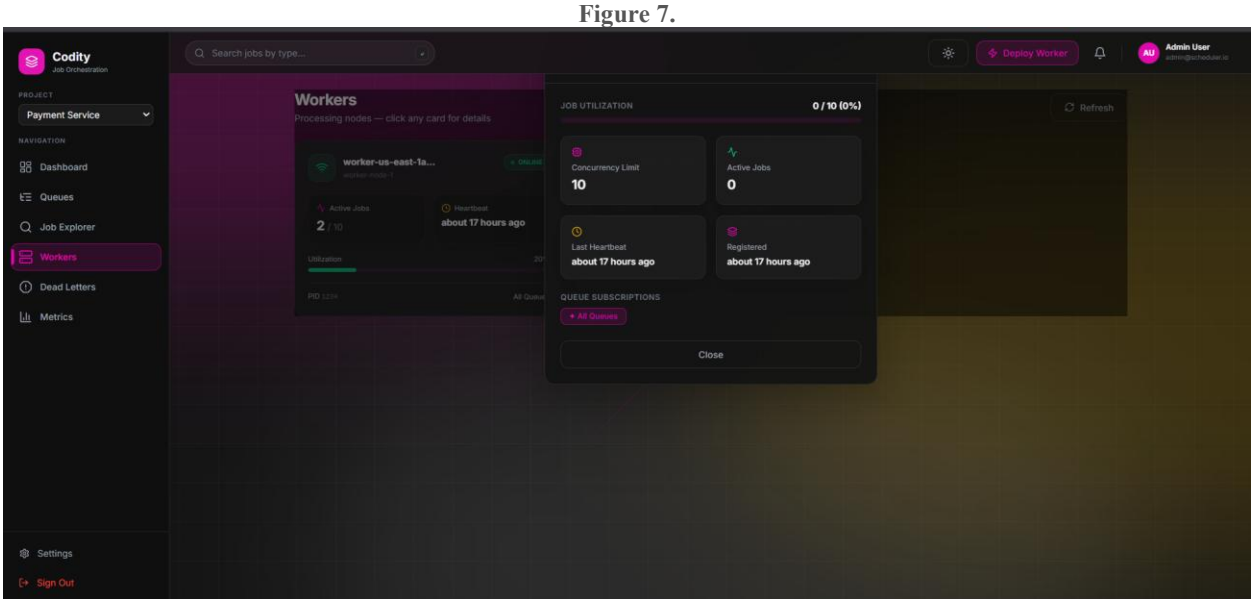

*Queue detail — pause/resume control*

**Figure 6.**



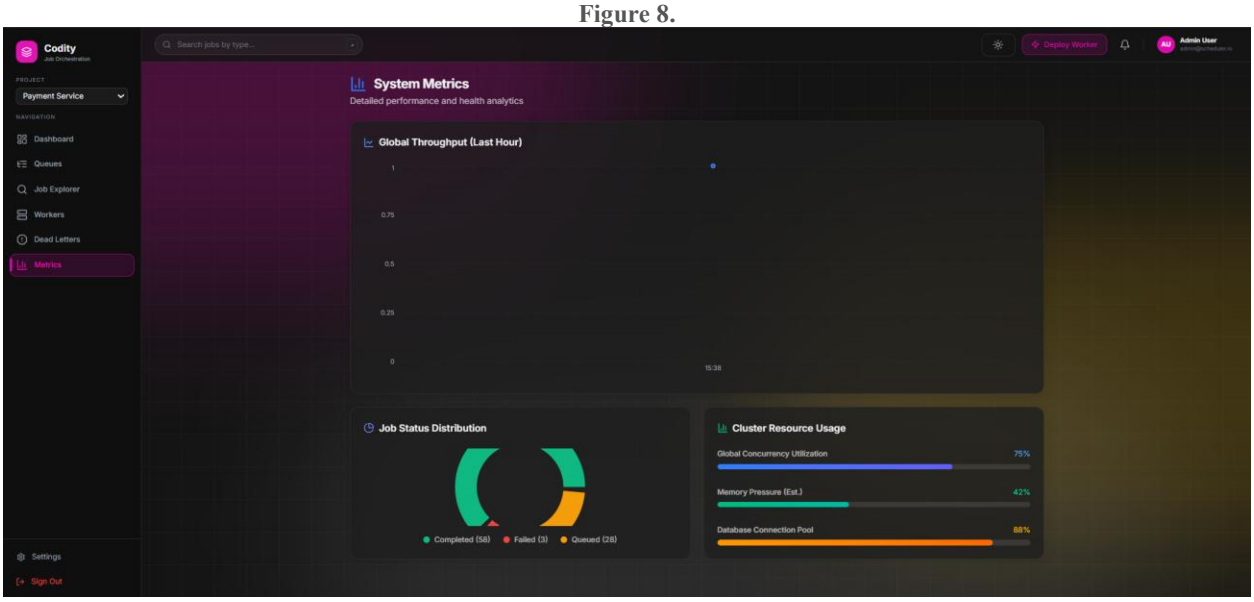

*Dead Letter Queue — entry detail with failure summary*

**Figure 7.**




*Workerspage — active worker with live heartbeat*

**Figure 8.**




*Metrics page — throughput and status distribution*

# 7. API Design

- Consistent REST resource structure: /projects, /projects/:id/queues, /queues/:id/jobs, /jobs/:id, mirroring the ownership hierarchy in the data model.
- Consistent JSON envelope for both success and error responses, with pagination metadata (page, limit, total, totalPages) on list endpoints.
- Worker-facing endpoints (claim/start/complete/fail) are kept path-compatible with the resource model while being routed through entirely separate authentication middleware — see Section 2.2.
- API versioning under /api/v1, so the worker and dashboard clients are insulated from breaking changes to future versions.

A full endpoint reference with example requests is included as part of the repository documentation (see Section 9); representative examples:

*POST /api/v1/queues/:queueId/jobs — create a job (immediate, delayed via scheduledAt, recurring via cronExpression, or as part of a dependency graph via dependsOn).*

*POST /api/v1/queues/:queueId/jobs/claim — worker-only, atomic claim (API-key auth).*

*POST /api/v1/jobs/:id/complete / /fail — worker-only, idempotent outcome reporting (API-key auth).*

# 8. Testing

Automated tests use Vitest and Supertest against a real database, covering:

- Authentication flow: registration, login, profile retrieval, invalid-credential rejection.
- Job lifecycle: creation, fetching within a queue.
- Worker execution end-to-end: claim → start (with idempotency check) → complete (idempotent, exact-row-count asserted) → exhausted retry → DEAD with a single Dead Letter Queue row (exact-count asserted).
- Queue lifecycle: pause prevents claiming; resume restores it.
- Authorization: an unauthenticated / cross-organization request is rejected with the correct status code rather than silently succeeding.

Test evidence is included as a screenshot of a full local test run below.

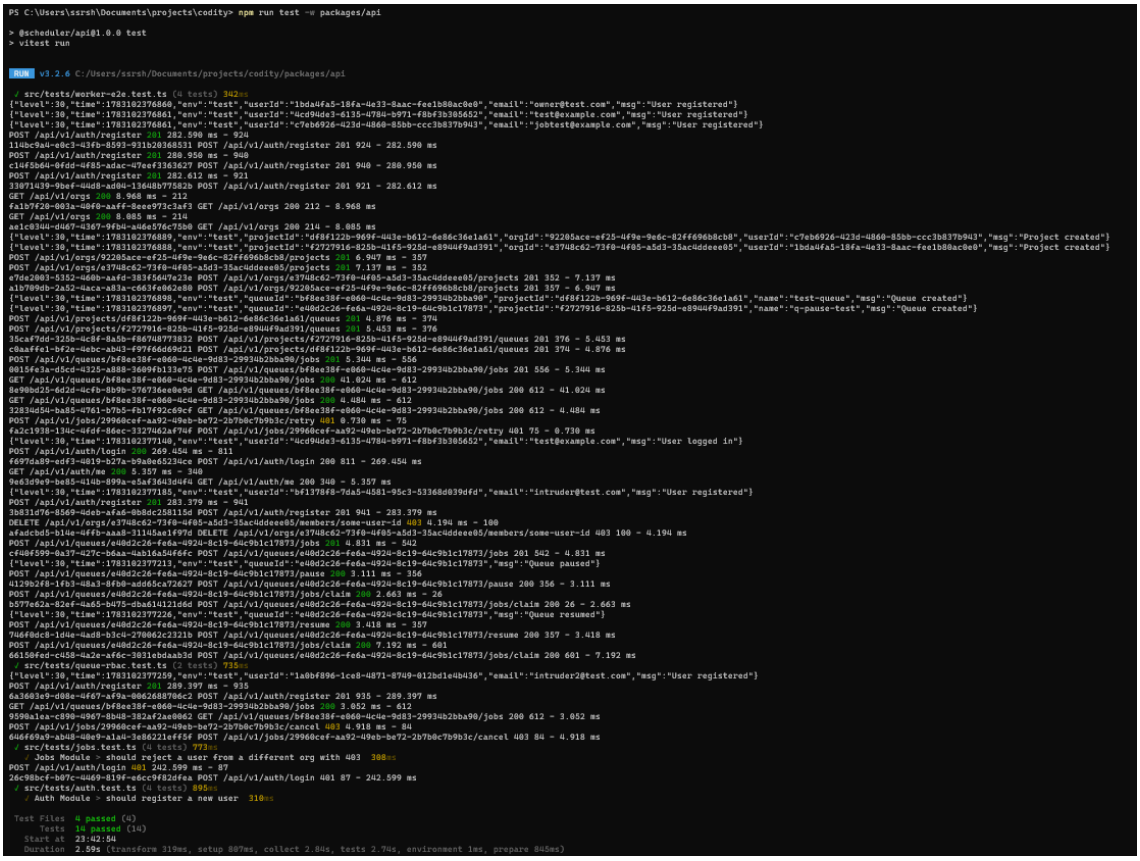



**Figure 9.** *Terminal output of npm run test -w packages/api, all suites passing.*

*Known gap: coverage is concentrated on the reliability-critical path (claim/start/complete/fail, retries, DLQ) rather than being exhaustive across every endpoint. This was a deliberate prioritization given time constraints — see Section 12.*

# 9. Deliverables & Design Decisions

## 9.1 Deliverables checklist

| Item | Status | Notes |
|---|---|---|
| Source code with setup instructions | Complete | README.md, repository root |
| Architecture diagram | Complete | Figure 1, Section 2 |
| ER diagram | Complete | Figure 2, Section 3 |
| API documentation | Complete | README.md / API reference; examples in Section 7 |
| Design decisions document | Complete | ARCHITECTURE.md, repository root |
| Automated tests for critical functionality | Complete | Section 8 |

## 9.2 Selected design decisions

**Advisory locks + FOR UPDATE SKIP LOCKED over an external queue broker:** chosen to keep the system self-contained (no Redis/RabbitMQ dependency) while still providing genuine atomic, contention-safe claiming. The tradeoff is that claim throughput is bounded by what a single PostgreSQL instance can serialize per queue — acceptable at the scale this assignment targets, and the first thing to revisit if throughput requirements grew significantly.

**Idempotent state transitions via conditional updates rather than external deduplication:** every claim/start/complete/fail call is safe to retry because the guard is baked into the SQL update itself (expected status + caller identity), not enforced by a separate idempotency-key cache. This was specifically hardened after an early version of the state machine allowed a duplicate report to silently double-write execution records.

**Separate authentication schemes for dashboard users and workers:** initially the worker and dashboard shared one JWT-based authentication path; this was deliberately re-architected mid-project after identifying that it created a fragile coupling — a change to cookie handling could silently break worker authentication and vice versa. Splitting them into structurally separate routing removed that coupling entirely rather than managing it with conditional logic.

**SameSite cookie policy adapts to deployment topology:** in the live cross-origin deployment (dashboard on Vercel, API on Render), authentication requests are routed through a same-origin proxy rewrite so the browser treats them as first-party, avoiding the cross-site cookie restrictions that privacy-focused browsers (Brave, Safari) apply by default. In a same-domain deployment (or local development) this proxy is unnecessary and cookies use a stricter policy directly.

**Worker service-account keys over shared JWTs:** workers authenticate with a dedicated, hashed, revocable API key rather than a JWT minted for a human user, so worker credentials can be rotated or revoked independently of any user account and never carry user-level permissions.

## 10. Bonus Features

| Feature | Status | Notes |
|---|---|---|
| Workflow dependencies | Implemented | JobDependency model; automatic resolution/cascade on completion or permanent failure |
| Rate limiting | Implemented | Separate tiers for login, general API, and worker polling traffic |
| Distributed locking | Implemented | PostgreSQL advisory locks, not an external lock service — see Section 9.2 |
| Queue sharding | Implemented | shardKey field with routing logic in the claim query |
| Event-driven execution | Implemented | Internal event bus decoupling state changes from WebSocket broadcast and dependency resolution |
| WebSocket live updates | Implemented | Project-scoped rooms, JWT-authenticated handshake, revocation-aware |
| Role-based access control | Implemented | Global roles + per-organization roles |
| AI-generated failure summaries | Implemented | Pattern-matched categorization (timeout/network/validation/unknown); see Section 12 for scope |

# 11. Setup Instructions & Deployment

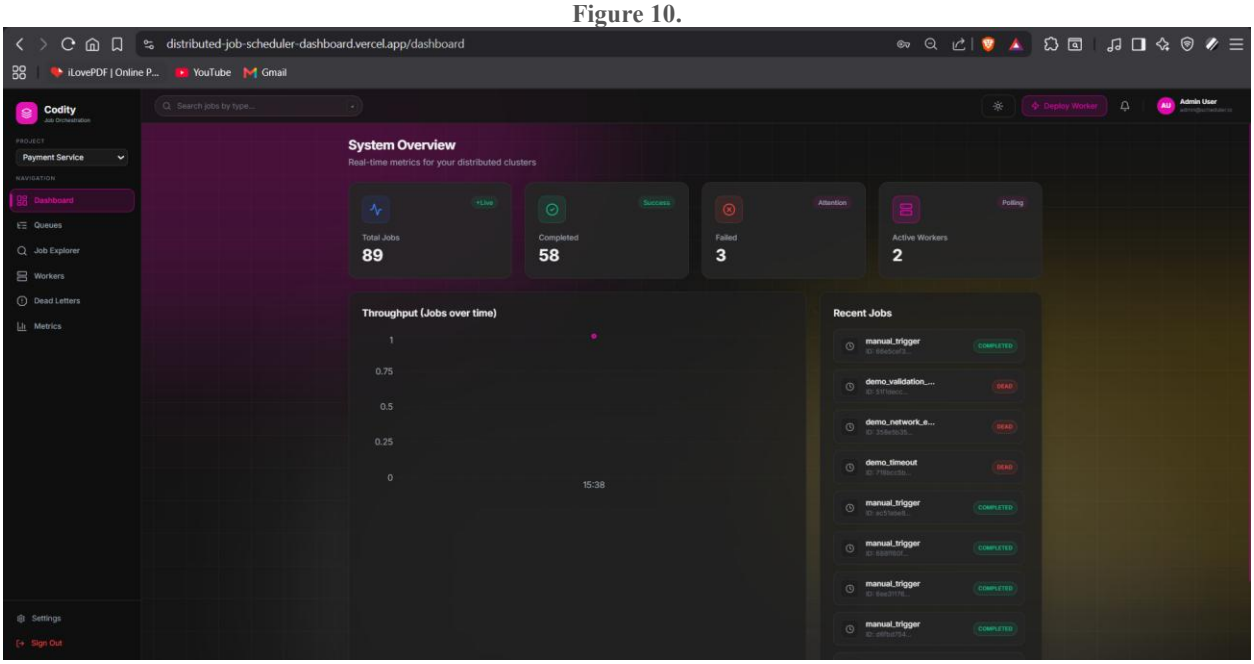
Full setup instructions (prerequisites, environment variables, migration, seeding, and running each package) are provided in README.md at the repository root. Summary:

- Prerequisites: Node.js, PostgreSQL, npm.
- Install: npm install (workspaces monorepo, installs all three packages).
- Configure: copy .env.example to .env and fill in DATABASE_URL, JWT_SECRET, JWT_REFRESH_SECRET, CORS_ORIGIN, FRONTEND_URL.
- Database: npm run db:migrate -w packages/api, then npm run db:seed -w packages/api — the seed script prints a worker API key needed by the worker process.
- Run: start the API, worker, and dashboard packages (see README.md for exact scripts); or use the provided Dockerfile / docker-compose.yml to bring up the full stack with one command.

**Live deployment:**

The system is also deployed live for convenience: the API and worker run as a single bundled Render Web Service (a free-tier accommodation, since Render's free plan does not support a persistent standalone background worker — in a paid or self-hosted deployment they would run as fully independent services, exactly as the codebase already supports), with PostgreSQL on Render's free plan and the dashboard on Vercel. See the title page for links. An external uptime monitor (UptimeRobot) periodically pings the API to reduce Render's free-tier idle spin-down during evaluation.

**Figure 10.**




*Live deployed dashboard, logged in, showing real data*

## 12. Known Limitations & Future Work

Listed explicitly rather than omitted, in line with the assignment's stated preference for engineering honesty over feature-count over-claiming.

- No stale-job reaper yet: if a worker process crashes between claiming a job and reporting its outcome, that job remains in RUNNING with no automatic watchdog to requeue or fail it. A heartbeat-timeout-based reaper is the natural next addition.
- No cross-worker global concurrency enforcement: each worker enforces its own local concurrency limit; there is no cluster-wide cap coordinated across multiple worker instances beyond what the queue-level claim query naturally serializes.
- No Prometheus-style metrics export or distributed tracing; observability currently consists of structured application logs and the in-dashboard metrics views.
- Job Explorer's list updates via a manual refresh control rather than reacting live to WebSocket events, unlike the toast-notification system elsewhere in the dashboard, which is real-time. A UX polish item, not a functional defect — the underlying data and event infrastructure required to fix it already exists and is used correctly on other pages.
- Mobile responsiveness has been implemented and lightly tested (sidebar collapse, scrollable tables, touch-friendly action buttons) but not exhaustively audited across the full range of devices and browsers.
- Free-tier deployment constraints: the live demo's Postgres instance is subject to Render's free-tier retention window, and the bundled worker/API process is a cost-driven accommodation rather than the target production topology (see Section 9.2).

## 13. Conclusion

The system implements the full core requirement set — authentication and multi-project queue management, five job creation modes, atomic and idempotent worker execution, a complete retry/DLQ lifecycle, and a live monitoring dashboard — plus all eight listed bonus features, on a normalized 16-model schema with concurrency-safe claiming verified by automated tests rather than asserted by description. The known limitations in Section 12 are the specific, honestly-scoped remainder rather than an exhaustive absence of polish, and each has a clear, stated path forward.